

Firewall Control

Understanding Linux Firewalling

You can use a firewall to limit traffic coming in to a server or going out of the server. Firewalling is implemented in the Linux kernel by means of the netfilter subsystem. Netfilter allows kernel modules to inspect every incoming, outgoing, or forwarded packet and act upon such a packet by either allowing it or blocking it. So, the kernel firewall allows for inspection of incoming packets, outgoing packets, and packets that are traversing from one interface to another if the RHEL server is providing routing functionality.

Understanding Previous Solutions

To interact with netfilter, different solutions can be used:

1. Iptables : On earlier versions of Red Hat Enterprise Linux, iptables was the default solution to configure netfilter packet filtering
2. FirewallD: On RHEL 7 and still on RHEL 8

Managing Firewall:

1. Firewall-cmd: command line utility
2. Firewall-config: Graphical utility

The firewallD Service

The firewallD service offers the same functionalities of the iptables tool and more. One of the new features of firewallD is zone-based firewalling. In a zone-based firewall, networks and interfaces are grouped into zones, with each zone configured with a different level of trust. The zones defined in firewallD are listed in The following Table, along with their default behavior for outgoing and incoming connections.

A zone is made up of a group of source network addresses and interfaces, plus the rules to process the packets that match those source addresses and network interfaces.

Zone	Outgoing Connections	Incoming Connections
drop	Allowed	Dropped.
block	Allowed	Rejected with an icmp-host-prohibited message.
public	Allowed	DHCPv6 client and SSH are allowed.
external	Allowed and masqueraded to the IP address of the outgoing network interface	SSH is allowed.
dmz	Allowed	SSH is allowed.
work	Allowed	DHCPv6 client, IPP and SSH are allowed.
home	Allowed	DHCPv6 client, multicast DNS, IPP, Samba client, and SSH are allowed.
internal	Allowed	Same as the home zone.
trusted	Allowed	Allowed.

Understanding FirewallD Services

The second key element while working with firewallD is the service. Note that a service in firewallD is *not* the

same as a service in systemd. In firewallld, some default services are defined, which allows administrators to easily allow or deny access to specific ports on a server. Behind each service is a configuration file that explains which UDP or TCP ports are involved, and if so required, which kernel modules must be loaded. To get a list of all services available on your computer, you can use the command **firewall-cmd --get-services**.

```
[root@localhost sssd]# firewall-cmd --get-services
```

```
amanda-client bacula bacula-client dhcp dhcpv6 dhcpv6-client dns ftp high-availability http https imaps ipp ipp-client  
ipsec Kerberos kpasswd ldap ldaps libvirt libvirt-tls mdns mountd ms-wbt mysql nfs ntp openvpn pmcd pmproxy  
pmwebapi pmwebapis pop3s postgresql proxy-dhcp radius rpc-bind samba samba-client smtp ssh telnet tftp tftp-client  
transmission-client vnc-server wbem-https
```

Contents of the ftp Service File

```
[root@server1 services]# cat ftp.xml
```

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<service>
```

```
<short>FTP</short>
```

```
<description>FTP is a protocol used for remote file transfer. If you plan to make your FTP server publicly available,  
enable this option. You need the vsftpd package installed for this option to be useful.</description>
```

```
<port protocol="tcp" port="21"/>
```

```
<module name="nf_conntrack_ftp"/>
```

```
</service>
```

Working with Firewall-cmd

As with the GUI tool, firewall-cmd can display all the available zones and switch to a different default zone. In the following example, the default zone is changed from the public to the internal zone:

```
# firewall-cmd --get-default-zone
```

```
public
```

```
# firewall-cmd --set-default-zone=internal
```

```
success
```

```
# firewall-cmd --get-default-zone
```

```
internal
```

```
#
```

The option **--list-all** is particularly useful. It lists all the configured interfaces and services allowed through a zone, as illustrated next:

```
# firewall-cmd --list-all internal
```

```
(default, active)
```

```
interfaces: eth0
```

```
sources:
```

```
services: samba-client ssh
```

```
ports:
masquerade: no
forward-ports:
icmp-blocks:
rich rules:
```

Interfaces are the system's names for hardware and virtual network adapters, as you can see in the above example. All active interfaces will be assigned to zones, either to the default zone or to a user-specified one. However, an interface cannot be assigned to more than one zone.

In its default configuration, firewalld pairs all interfaces with the public zone and doesn't set up sources for any zones. As a result, public is the only active zone.

Sources are incoming IP address ranges, which also can be assigned to zones. A source (or overlapping sources) cannot be assigned to multiple zones. Doing so results in undefined behavior, as it would not be clear which rules should be applied to that source.

Since specifying a source is not required, for every packet there will be a zone with a matching interface, but there won't necessarily be a zone with a matching source. This indicates some form of precedence with priority going to the more specific source zones, but more on that later. First, let's inspect how the public zone is configured:

```
# firewall-cmd --zone=public --list-all
public (default, active)
  interfaces: eno1 eno2
  sources:
  services: dhcpv6-client ssh
  ports:
  masquerade: no
  forward-ports:
  icmp-blocks:
  rich rules:
# firewall-cmd --permanent --zone=public --get-target
default
```

Going line by line through the output:

- `public (default, active)` indicates that the public zone is the default zone (interfaces default to it when they come up), and it is active because it has at least one interface or source associated with it.
- `interfaces: eno1 eno2` lists the interfaces associated with the zone.
- `sources:` lists the sources for the zone. There aren't any now, but if there were, they would be of the form `xxx.xxx.xxx.xxx/xx`.
- `services: dhcpv6-client ssh` lists the services allowed through the firewall. You can get an exhaustive list of firewalld's defined services by executing `firewall-cmd --get-services`.
- `ports:` lists port destinations allowed through the firewall. This is useful if you need to allow a service that isn't defined in firewalld.
- `masquerade: no` indicates that IP masquerading is disabled for this zone. If enabled, this would allow IP forwarding, with your computer acting as a router.

- `forward-ports`: lists ports that are forwarded.
- `icmp-blocks`: a blacklist of blocked icmp traffic.
- `rich rules`: advanced configurations, processed first in a zone.
- `default` is the target of the zone, which determines the action taken on a packet that matches the zone yet isn't explicitly handled by one of the above settings.

As with many of the `firewall-cmd` command options, the default zone is assumed if no zone is specified with the `--zone` command switch. You can add and remove ports and services from a zone with the `--add-port`, `--add-service`, `--remove-port`, and `--remove-services` switches, respectively. The next example enables the http service for traffic hitting the dmz zone:

```
# firewall-cmd --zone=dmz --add-service=http
success
```

By default, all configuration changes made by `firewall-cmd` do not survive a server reboot. To make a change that survives a reboot, add the `--permanent` switch to `firewall-cmd`. Then, run `firewall-cmd --reload` to implement the change immediately.

A Simple Single-Zoned Example

Say you just want to lock down your firewall. Simply remove the services currently allowed by the public zone and reload:

```
# firewall-cmd --permanent --zone=public --remove-service=dhcpv6-client
# firewall-cmd --permanent --zone=public --remove-service=ssh
# firewall-cmd --reload
```

These commands result in the following firewall:

```
# firewall-cmd --zone=public --list-all
public (default, active)
  interfaces: eno1 eno2
  sources:
  services:
  ports:
  masquerade: no
  forward-ports:
  icmp-blocks:
  rich rules:
# firewall-cmd --permanent --zone=public --get-target
default
```

In the spirit of keeping security as tight as possible, if a situation arises where you need to open a temporary hole in your firewall (perhaps for ssh), you can add the service to just the current session (omit `--permanent`) and instruct `firewalld` to revert the modification after a specified amount of time:

```
# firewall-cmd --zone=public --add-service=ssh --timeout=5m
```

The timeout option takes time values in seconds (s), minutes (m) or hours (h).

Targets

When a zone processes a packet due to its source or interface, but there is no rule that explicitly handles the packet, the target of the zone determines the behavior:

- `ACCEPT`: accept the packet.
- `%%REJECT%%`: reject the packet, returning a reject reply.
- `DROP`: drop the packet, returning no reply.
- `default`: don't do anything. The zone washes its hands of the problem, and kicks it "upstairs".

Precedence

Active zones fulfill two different roles. Zones with associated interface(s) act as interface zones, and zones with associated source(s) act as source zones (a zone could fulfill both roles). Firewallld handles a packet in the following order:

1. The corresponding source zone. Zero or one such zones may exist. If the source zone deals with the packet because the packet satisfies a rich rule, the service is whitelisted, or the target is not default, we end here. Otherwise, we pass the packet on.
2. The corresponding interface zone. Exactly one such zone will always exist. If the interface zone deals with the packet, we end here. Otherwise, we pass the packet on.
3. The firewallld default action. Accept icmp packets and reject everything else.

The take-away message is that source zones have precedence over interface zones. Therefore, the general design pattern for multi-zoned firewallld configurations is to create a privileged source zone to allow specific IP's elevated access to system services and a restrictive interface zone to limit the access of everyone else.

A Simple Multi-Zoned Example

To demonstrate precedence, let's swap ssh for http in the public zone and set up the default internal zone for our favorite IP address, 1.1.1.1. The following commands accomplish this task:

```
# firewall-cmd --permanent --zone=public --remove-service=ssh
# firewall-cmd --permanent --zone=public --add-service=http
# firewall-cmd --permanent --zone=internal --add-source=1.1.1.1
# firewall-cmd --reload
```

which results in the following configuration:

```
# firewall-cmd --zone=public --list-all
public (default, active)
  interfaces: eno1 eno2
  sources:
  services: dhcpv6-client http
  ports:
  masquerade: no
  forward-ports:
  icmp-blocks:
```

```

    rich rules:
# firewall-cmd --permanent --zone=public --get-target
default
# firewall-cmd --zone=internal --list-all
internal (active)
    interfaces:
    sources: 1.1.1.1
    services: dhcpv6-client mdns samba-client ssh
    ports:
    masquerade: no
    forward-ports:
    icmp-blocks:
    rich rules:
# firewall-cmd --permanent --zone=internal --get-target
default

```

With the above configuration, if someone attempts to `ssh` in from 1.1.1.1, the request would succeed because the source zone (`internal`) is applied first, and it allows `ssh` access.

If someone attempts to `ssh` from somewhere else, say 2.2.2.2, there wouldn't be a source zone, because no zones match that source. Therefore, the request would pass directly to the interface zone (`public`), which does not explicitly handle `ssh`. Since `public`'s target is `default`, the request passes to the `firewalld` default action, which is to reject it.

What if 1.1.1.1 attempts `http` access? The source zone (`internal`) doesn't allow it, but the target is `default`, so the request passes to the interface zone (`public`), which grants access.

Now let's suppose someone from 3.3.3.3 is trolling your website. To restrict access for that IP, simply add it to the preconfigured drop zone, aptly named because it drops all connections:

```

# firewall-cmd --permanent --zone=drop --add-source=3.3.3.3
# firewall-cmd --reload

```

The next time 3.3.3.3 attempts to access your website, `firewalld` will send the request first to the source zone (`drop`). Since the target is `DROP`, the request will be denied and won't make it to the interface zone (`public`) to be accepted.

Configuring Firewalld Rich Rules

you have worked with `firewalld` services. Although convenient, the options that are offered by `firewalld` services are sometimes a bit limited. That is why `firewalld` offers alternative solutions for allowing traffic as well. Currently, there are two solutions:

- Direct rules allow administrators to add detailed hand code rules into the firewall configuration. They offer advanced features and are somewhat hard to manage. They use a syntax that looks a lot like `iptables` syntax and allow administrators to work with `firewalld` in an `iptables`-like way.
- Rich rules give administrators an easy-to-use expressive language to define custom rules that cannot be created using the default `firewalld` syntax.

Rich Rule Syntax

Rich rules are used to create allow/deny rules, but with advanced options, such as the following:

- Logging configuration

- Port forwarding
- Masquerading
- Rate limiting
- Allow/deny connections for one specific zone

The basic syntax of a rich rule is as follows:

- Rule
- [source] [destination]
- {service|port|protocol|icmp-block|masquerade|forward-port}
- [log] [audit]
- [accept|reject|drop]

Rich Rules Options:

--add-rich-rule=<RULE>	Adds <RULE> to the default zone or to the zone that is specified.
--remove-rich-rule=<RULE>	Removes <RULE> from the default zone or from the zone that is specified.
--query-rich-rule=<RULE>	Queries if <RULE> has been added to a zone. Returns 0 if the rule is present and 1 if it is not and does not give any further details.
--list-rich-rules	Lists all rich rules for the default zone or for the zone that is specified as an argument.

Examples Of Rich Rules:

Allow Telnet connection only from 192.168.1.101/24. Drop Telnet connection from remaining hosts:

```
firewall-cmd --add-rich-rule='rule family=ipv4 source address=192.168.1.101/32 service name=telnet accept'
```

Allow FTP connection only from 192.168.1.2/24. Reject FTP connections from remaining systems.

```
firewall-cmd --add-rich-rule='rule family=ipv4 source address=192.168.1.2/32 port port=21 protocol=tcp accept'
```

Reject ping requests from all hosts with error message.

```
firewall-cmd --add-rich-rule='rule protocol value=icmp reject'
```

Configuring Network Address Translation

Network Address Translation (NAT) is a common firewalld functionality. To use NAT, the RHEL 7 machine needs to be configured as a router. Therefore, it needs to have at least two network interfaces and forward incoming packets from one interface to the other interface. To configure a server as a router, make sure to set the sysctl tuneable **net.ipv4.ip_forward = 1**.

Then you can do static routing through port forwarding to another server in the same port or another port, you can do it with **--add-forward-port** which applied on every packet incoming to your server no matter what the source of this packets or through rich rules to customize the values of when traffic's source address is x forward to the server y on port P.

To configure masquerading for hosts in a specific zone, use the following command:

```
firewall-cmd --permanent --zone=<ZONE> --add-masquerade
```

Let's do it with port forwarding

If you want to forward all packets coming in at port 2022 on the NAT router to an SSH process on internal host 10.0.0.10, for example, you need the following command:

```
firewall-cmd --permanent --zone=public --add-forwardport=port=2022:proto=tcp:toport=22:toaddr=10.0.0.10
```

The field toaddr= could be empty if you want to do portforwarding locally not to another server

We can do this by rich rules like this:

```
firewall-cmd --permanent --zone=<ZONE> --add-masquerade
```

The NAT is active for the entire zone or we customize source traffic NAT is active for:

```
firewall-cmd --permanent --zone=<ZONE> --add-rich-rule='rule family=ipv4 source address=10.0.0.0/24 masquerade'
```

A Practical Multi-Zoned Example

Suppose you are setting up a firewall for a server at your organization. You want the entire world to have http and https access, your organization (1.1.0.0/16) and workgroup (1.1.1.0/8) to have ssh access, and your workgroup to have samba access. Using zones in firewalld, you can set up this configuration in an intuitive manner.

Given the naming, it seems logical to commandeer the public zone for your world-wide purposes and the internal zone for local use. Start by replacing the dhcpv6-client and ssh services in the public zone with http and https:

```
# firewall-cmd --permanent --zone=public --remove-service=dhcpv6-client
# firewall-cmd --permanent --zone=public --remove-service=ssh
# firewall-cmd --permanent --zone=public --add-service=http
# firewall-cmd --permanent --zone=public --add-service=https
```

Then trim mdns, samba-client and dhcpv6-client out of the internal zone (leaving only ssh) and add your organization as the source:

```
# firewall-cmd --permanent --zone=internal --remove-service=mdns
# firewall-cmd --permanent --zone=internal --remove-service=samba-client
# firewall-cmd --permanent --zone=internal --remove-service=dhcpv6-client
# firewall-cmd --permanent --zone=internal --add-source=1.1.0.0/16
```

To accommodate your elevated workgroup samba privileges, add a rich rule:

```
# firewall-cmd --permanent --zone=internal --add-rich-rule='rule
↳family=ipv4 source address="1.1.1.0/8" service name="samba"
↳accept'
```

Finally, reload, pulling the changes into the active session:

```
# firewall-cmd --reload
```

Only a few more details remain. Attempting to `ssh` in to your server from an IP outside the internal zone results in a reject message, which is the `firewalld` default. It is more secure to exhibit the behavior of an inactive IP and instead drop the connection. Change the public zone's target to `DROP` rather than `default` to accomplish this:

```
# firewall-cmd --permanent --zone=public --set-target=DROP
# firewall-cmd --reload
```

But wait, you no longer can ping, even from the internal zone! And `icmp` (the protocol ping goes over) isn't on the list of services that `firewalld` can whitelist. That's because `icmp` is an IP layer 3 protocol and has no concept of a port, unlike services that are tied to ports. Before setting the public zone to `DROP`, pinging could pass through the firewall because both of your `default` targets passed it on to the `firewalld` default, which allowed it. Now it's dropped.

To restore pinging to the internal network, use a rich rule:

```
# firewall-cmd --permanent --zone=internal --add-rich-rule='rule
↳protocol value="icmp" accept'
# firewall-cmd --reload
```

In summary, here's the configuration for the two active zones:

```
# firewall-cmd --zone=public --list-all
public (default, active)
  interfaces: eno1 eno2
  sources:
  services: http https
  ports:
  masquerade: no
  forward-ports:
  icmp-blocks:
  rich rules:
# firewall-cmd --permanent --zone=public --get-target
DROP
# firewall-cmd --zone=internal --list-all
internal (active)
  interfaces:
  sources: 1.1.0.0/16
  services: ssh
  ports:
  masquerade: no
  forward-ports:
  icmp-blocks:
  rich rules:
    rule family=ipv4 source address="1.1.1.0/8"
    ↳service name="samba" accept
    rule protocol value="icmp" accept
# firewall-cmd --permanent --zone=internal --get-target
default
```

Eng. Muhammad Adel